

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Microcontrollers

(CO3117)

Report Lab 3

BUTTON/SWITCH

Advisor: TS. Lê Trọng Nhân

Student: Nguyễn Chí Thanh 2313078

Ho Chi Minh City, September 23, 2025

Mục lục

1	Introduction	4
2	Exercise and Report	4
2.1	Exercise 1: Sketch an FSM	4
2.2	Exercise 2: Proteus Schematic	5
2.3	Exercise 3: Create STM32 Project	7
2.4	Exercise 4: Modify Timer Parameters	7
2.5	Exercise 5: Adding code for button debouncing	10
2.6	Exercise 6: Adding code for displaying modes	13
2.7	Exercise 7: Adding code for increasing time duration value for the red LEDs	16
2.8	Exercise 8: Adding code for increasing time duration value for the amber LEDs	17
2.9	Exercise 9: Adding code for increasing time duration value for the green LEDs	18
2.10	Exercise 10: To finish the project	19

1 Introduction

In this lab, we will explore various LED animations using a microcontroller. The goal is to understand how to control LEDs and create visually appealing effects. This report consists of:

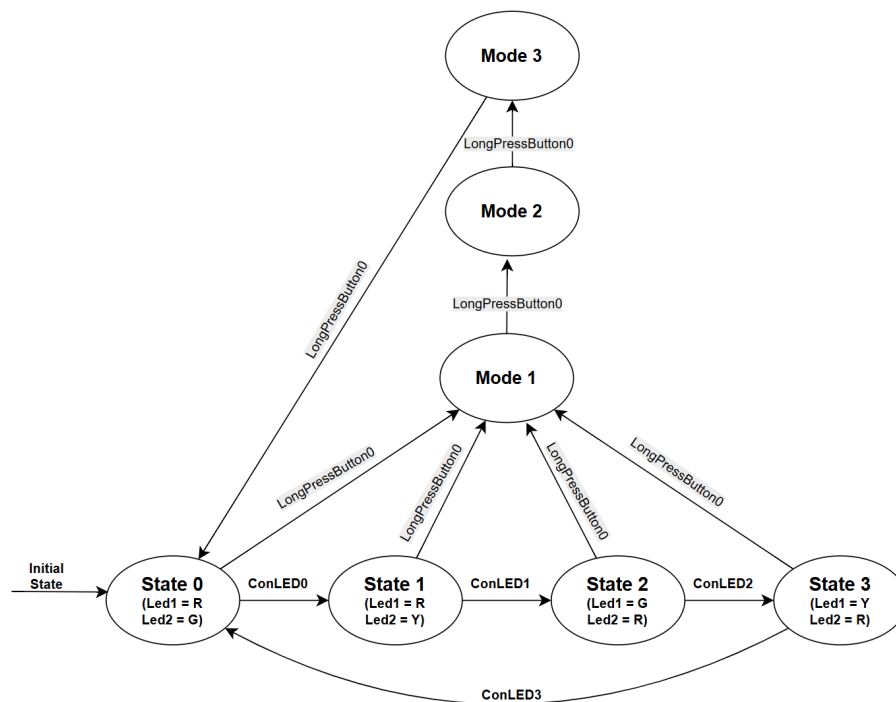
- Source code C language for LED animations in STM32CubeIDE.
- Simulate LED animations using Proteus software.

For more details about source code, visit my [GitHub repository](#).

2 Exercise and Report

2.1 Exercise 1: Sketch an FSM

Your task in this exercise is to sketch an FSM that describes your idea of how to solve the problem



Hình 2.1: FSM Diagram for LED Control System

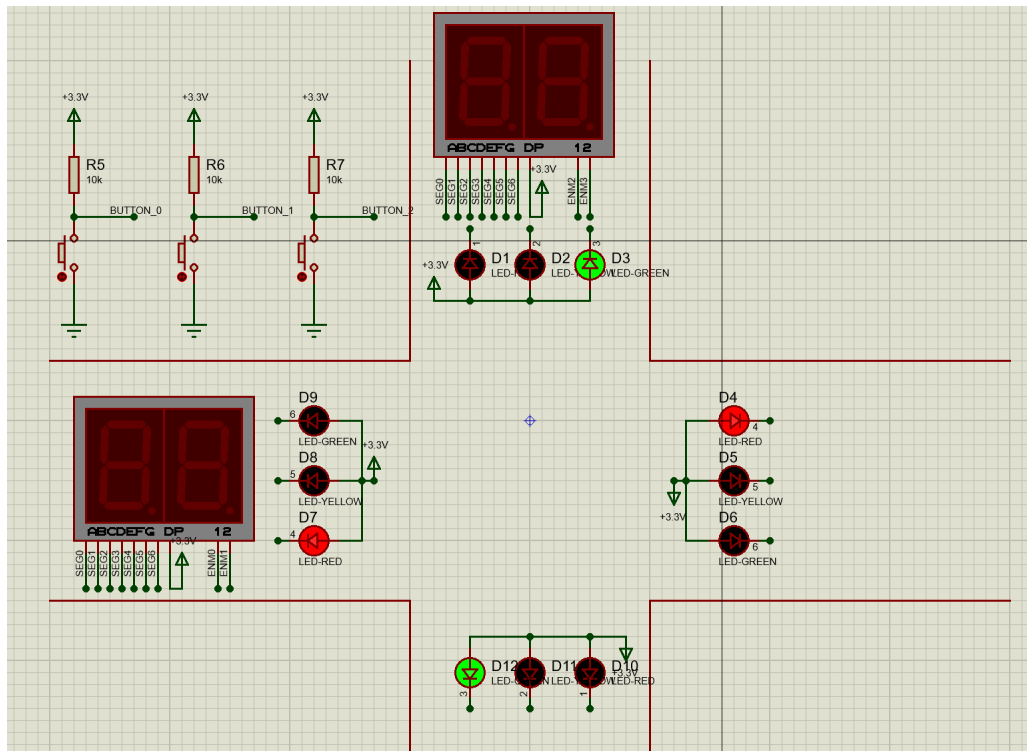
Bài báo cáo trình bày FSM bao gồm 2 chế độ chính là chế độ tự động (Automatic Mode) và chế độ điều chỉnh (Config Mode). Trong đó:

- Chế độ tự động (Automatic Mode hay mode 0): Hệ thống sẽ tự động chuyển đổi giữa 4 trạng thái của đèn LED (Đỏ, Vàng, Xanh) theo thời gian đã được cài đặt sẵn và điều chỉnh bộ đếm hiển thị led 7 đoạn. Nếu nhận tín hiệu longpress từ button0 thì trạng thái sẽ chuyển sang mode 1 của config mode. Các tín hiệu nút còn lại không làm thay đổi gì.
- Chế độ điều chỉnh (Config Mode): Người dùng có thể điều chỉnh thời gian tối đa của các đèn LED (mode 1 : RED, mode 2: AMBER, mode 3: GREEN) và hiển thị giá trị này trên led 7 đoạn bằng cách sử dụng các nút bấm:
 - Longpress button0 để chuyển mode tiếp theo.
 - Press button0 để giảm thời gian.
 - Press button1 để tăng thời gian.
 - LongPress button1 để tăng nhanh.
 - Press button2 sẽ lưu giá trị thời gian tối đa đã được điều chỉnh cho đèn LED hiện tại.

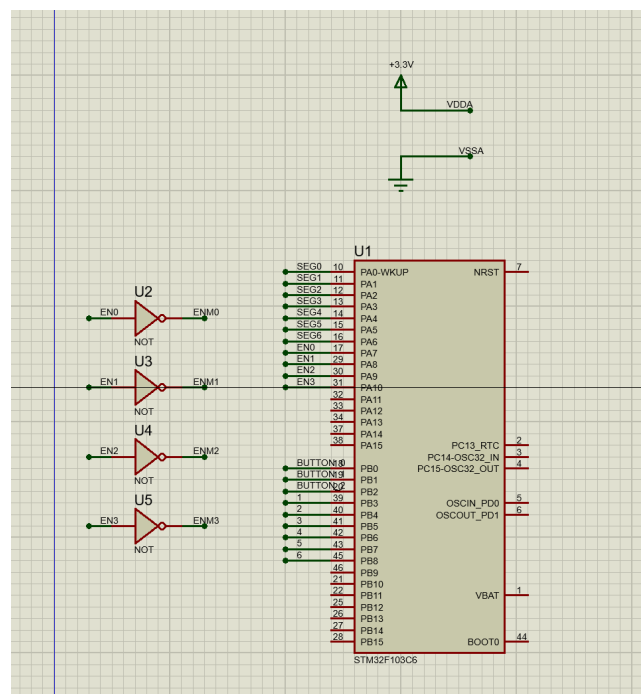
Bài báo cáo quy ước thời gian của đèn đỏ sẽ bằng tổng thời gian của vàng và xanh của lần khác. Nếu thời gian đèn đỏ $\leq$ đèn vàng thì led sẽ hiển thị nhấp nháy.

2.2 Exercise 2: Proteus Schematic

Your task in this exercise is to draw a Proteus schematic for the problem above.



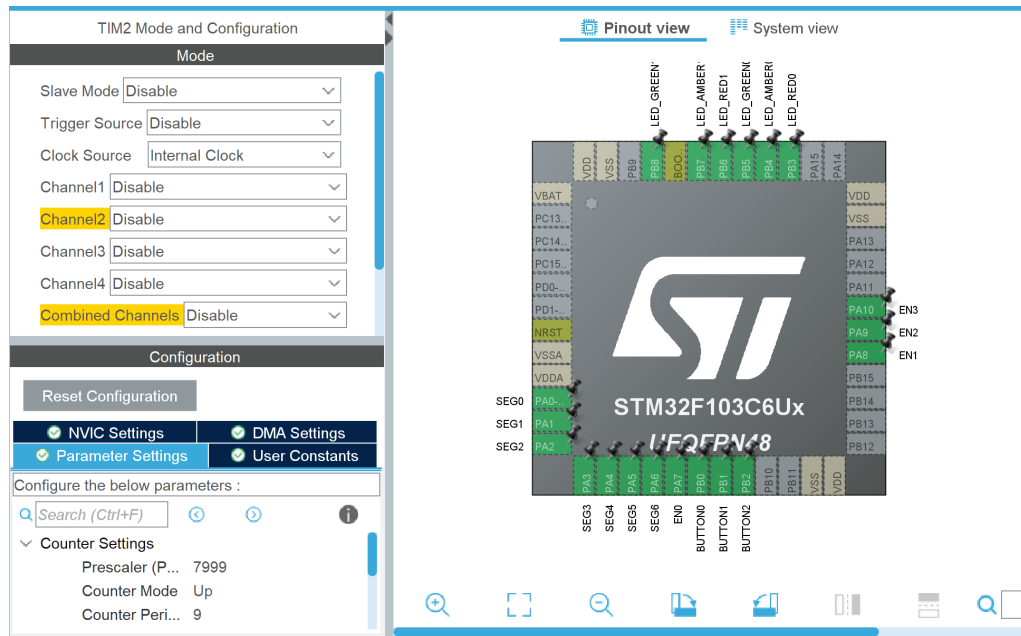
Hình 2.2: Proteus Schematic for LED Control System



Hình 2.3: Proteus Schematic for LED Control System

2.3 Exercise 3: Create STM32 Project

Your task in this exercise is to create a project that has pin corresponding to the Proteus schematic that you draw in previous section. You need to set up your timer interrupt is about 10ms.



Hình 2.4: STM32CubeMX Configuration

2.4 Exercise 4: Modify Timer Parameters

Your task in this exercise is to modify the timer settings so that when we want to change the time duration of the timer interrupt, we change it the least and it will not affect the overall system. For example, the current system we have implemented is that it can blink an LED in 2 Hz, with the timer interrupt duration is 10ms. However, when we want to change the timer interrupt duration to 1ms or 100ms, it will not affect the 2Hz blinking LED.

Listing 1: Define macro and timer variable

```
#define MAX_SOFTWARE_TIMER 10
#define TIMER_CYCLE 10 // Timer interrupt period is 10ms

typedef struct {
    int counter;
    uint8_t flag;
    int next_index;
} SoftwareTimer;
```

```

extern SoftwareTimer software_timers[MAX_SOFTWARE_TIMER];

void setTimer(int duration, int index);
int istimer_flag(int index);
void timer_run();

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim);

```

Listing 2: Initialize timer variable and function

```

SoftwareTimer software_timers[MAX_SOFTWARE_TIMER] = {0};
int head_timer = -1;
void setTimer(int index, int duration){
    int count = duration/TIMER_CYCLE;
    if(index >= MAX_SOFTWARE_TIMER) return;
    software_timers[index].flag = 0;
    if(head_timer == -1){
        head_timer = index;
        software_timers[index].counter = count;
        software_timers[index].next_index = -1;
        return;
    }
    else{
        if(software_timers[head_timer].counter > count){
            software_timers[head_timer].counter -= count;
            software_timers[index].next_index = head_timer;
            head_timer = index;
            software_timers[index].counter = count;
            return;
        }
        else{
            count -= software_timers[head_timer].counter;
            int pre = head_timer;
            int temp = software_timers[head_timer].next_index;
            while(temp != -1){
                if(count >= software_timers[temp].counter){
                    count -= software_timers[temp].counter;
                    pre = temp;
                }
                temp = software_timers[temp].next_index;
            }
            head_timer = pre;
            software_timers[head_timer].counter -= count;
            software_timers[index].next_index = head_timer;
            head_timer = index;
            software_timers[index].counter = count;
            return;
        }
    }
}

```

```

        temp = software_timers[temp].next_index;
    }
    else{
        software_timers[temp].counter -= count;
        software_timers[index].next_index = temp;
        software_timers[pre].next_index = index;
        software_timers[index].counter = count;
        return;
    }
}
software_timers[pre].next_index = index;
software_timers[index].next_index = -1;
software_timers[index].counter = count;
return;
}
}

void timer_run(){
    if(head_timer == -1) return;
    software_timers[head_timer].counter--;
    while(software_timers[head_timer].counter <= 0){
        software_timers[head_timer].flag = 1;
        head_timer = software_timers[head_timer].next_index;
    }
}

int istimer_flag(int timer_index){
    if(timer_index >= MAX_SOFTWARE_TIMER) return 0;
    if(software_timers[timer_index].flag == 1) {
        software_timers[timer_index].flag = 0;
        return 1;
    }
    return 0;
}

```

Listing 3: Timer interrupt callback function

```

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){

```

```
if(htim->Instance == TIM2){
    button_reading();
    timer_run();
}
}
```

2.5 Exercise 5: Adding code for button debouncing

Following the example of button reading and debouncing in the previous section, your tasks in this exercise are:

- To add new files for input reading and output display.
- To add code for button debouncing.
- To add code for increasing mode when the first button is pressed.

Listing 4: Define macro and button variable

```
#define MAX_BUTTONS 3
#define TIME_BOUNCING 10 //ms
#define TIME_BUTTON_SCAN 10 //ms
#define TIME_OUT_LONG_PRESS 500 //ms

#define BUTTON_IS_PRESSED GPIO_PIN_RESET
#define BUTTON_IS_RELEASED GPIO_PIN_SET

enum ButtonState {
    BUTTON_RELEASED,
    BUTTON_PRESSED,
    BUTTON_LONG_PRESSED
};

extern enum ButtonState buttonState[MAX_BUTTONS];
extern uint8_t flagPress[MAX_BUTTONS];
extern uint8_t flagLongPress[MAX_BUTTONS];

extern uint16_t BUTTON_MAP[MAX_BUTTONS];
extern GPIO_TypeDef* BUTTON_PORT_MAP[MAX_BUTTONS];
```

```

extern GPIO_PinState buttonBuffer[MAX_BUTTONS];
extern GPIO_PinState debounceButtonBuffer[MAX_BUTTONS];
extern uint16_t counterButtonBuffer[MAX_BUTTONS];
extern uint8_t flagButtonLongPress[MAX_BUTTONS];
extern uint16_t counterButtonLongPress[MAX_BUTTONS];
extern uint8_t flagTransitionLongPress[MAX_BUTTONS];

```

Listing 5: Define and initialize button variable

```

uint16_t BUTTON_MAP[MAX_BUTTONS] = {BUTTON0_Pin, BUTTON1_Pin, BUTTON2_Pin};
GPIO_TypeDef* BUTTON_PORT_MAP[MAX_BUTTONS] = {BUTTON0_GPIO_Port, BUTTON1_GPIO_Port,
    BUTTON2_GPIO_Port};
GPIO_PinState buttonBuffer[MAX_BUTTONS];
GPIO_PinState debounceButtonBuffer[MAX_BUTTONS];
uint16_t counterButtonBuffer[MAX_BUTTONS] = {0};
uint8_t flagButtonLongPress[MAX_BUTTONS] = {0};
uint16_t counterButtonLongPress[MAX_BUTTONS] = {0};
uint8_t flagPress[MAX_BUTTONS] = {0};
uint8_t flagLongPress[MAX_BUTTONS] = {0};
uint8_t flagTransitionLongPress[MAX_BUTTONS] = {0};
enum ButtonState buttonState[MAX_BUTTONS];

void button_reading(void){
    for(int i = 0; i < MAX_BUTTONS; i++){
        GPIO_PinState temp = HAL_GPIO_ReadPin(BUTTON_PORT_MAP[i], BUTTON_MAP[i]);
        if(debounceButtonBuffer[i] != temp) counterButtonBuffer[i] = 0;
        else{
            if(counterButtonBuffer[i] < TIME_BOUNCING) counterButtonBuffer[i] +=
                TIME_BUTTON_SCAN;
            else{
                if(buttonBuffer[i] == temp && temp == BUTTON_IS_RELEASED) continue;
                buttonBuffer[i] = temp;
                if(buttonBuffer[i] == BUTTON_IS_RELEASED){
                    flagButtonLongPress[i] = 0;
                    counterButtonLongPress[i] = 0;
                }
                else{
                    if(counterButtonLongPress[i] < TIME_OUT_LONG_PRESS)
                        counterButtonLongPress[i] += TIME_BUTTON_SCAN;
                }
            }
        }
    }
}

```

```

        else flagButtonLongPress[i] = 1;
    }
}

    }
    }
    debounceButtonBuffer[i] = temp;
}
}

unsigned char is_button_pressed(int index){
    if(index >= MAX_BUTTONS) return 0;
    return (buttonBuffer[index] == BUTTON_IS_PRESSED);
}

unsigned char is_button_long_pressed(int index){
    if(index >= MAX_BUTTONS) return 0xff;
    return (flagButtonLongPress[index] == 1);
}

void fsm_button(void){
    for(int i = 0; i < MAX_BUTTONS; i++){
        switch (buttonState[i]) {
            case BUTTON_RELEASED:
                if (is_button_pressed(i)) {
                    buttonState[i] = BUTTON_PRESSED;
                    flagPress[i] = 1;
                }
                break;
            case BUTTON_PRESSED:
                if (!is_button_pressed(i)) buttonState[i] = BUTTON_RELEASED;
                else if (is_button_long_pressed(i)){
                    flagTransitionLongPress[i] = 1;
                    buttonState[i] = BUTTON_LONG_PRESSED;}
                break;
            case BUTTON_LONG_PRESSED:
                if (!is_button_pressed(i)) buttonState[i] = BUTTON_RELEASED;
                flagLongPress[i] = 1;
                break;

```

```

    }

}

}

```

2.6 Exercise 6: Adding code for displaying modes

Your tasks in this exercise are:

- To add code for display mode on seven-segment LEDs, and
- To add code for blinking LEDs depending on the mode that is selected.

Listing 6: Define and initialize display variable

```

int index_led = 0;
int led_buffer[MAX_LED] = {0};

void display7SEG(int num){
    if(num > 9 || num < 0) return;
    HAL_GPIO_WritePin(SEG0_GPIO_Port, SEG0_Pin, num==1 || num==4);
    HAL_GPIO_WritePin(SEG1_GPIO_Port, SEG1_Pin, num==5 || num==6);
    HAL_GPIO_WritePin(SEG2_GPIO_Port, SEG2_Pin, num==2);
    HAL_GPIO_WritePin(SEG3_GPIO_Port, SEG3_Pin, num==1 || num==4 || num==7);
    HAL_GPIO_WritePin(SEG4_GPIO_Port, SEG4_Pin, !(num == 0 || num == 2 || num == 6
        || num == 8));
    HAL_GPIO_WritePin(SEG5_GPIO_Port, SEG5_Pin, num==1 || num==2 || num==3 ||
        num==7);
    HAL_GPIO_WritePin(SEG6_GPIO_Port, SEG6_Pin, num== 0 || num==1 || num==7);
}

uint16_t ENMAP[MAX_LED] = {EN0_Pin, EN1_Pin, EN2_Pin, EN3_Pin};

void update7SEG(int index){
    if(index >= MAX_LED || index < 0) return;
    HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin | EN1_Pin | EN2_Pin | EN3_Pin,
        SET);
    display7SEG(led_buffer[index]);
}

```

```

        HAL_GPIO_WritePin(ENO_GPIO_Port, ENMAP[index], RESET);
    }

#define TRAFIC_SCAN 50 //ms

int TempConfigure;

int32_t TIME_RED_LED = 5000/TRAFIC_SCAN;
int32_t TIME_AMBER_LED = 2000/TRAFIC_SCAN;
int32_t TIME_GREEN_LED = 3000/TRAFIC_SCAN;

int32_t counter_traffic_light = 0;
int8_t state_traffic_light = -1;
int32_t counter_7LED = 0;

void traffic_light(){
//CONTROL LED TRAFFIC
    if(counter_traffic_light == 0){
        HAL_GPIO_WritePin(LED_RED0_GPIO_Port, LED_RED0_Pin, 0);
        HAL_GPIO_WritePin(LED_GREEN0_GPIO_Port, LED_GREEN0_Pin, 1);
        HAL_GPIO_WritePin(LED_AMBER0_GPIO_Port, LED_AMBER0_Pin, 1);
        HAL_GPIO_WritePin(LED_RED1_GPIO_Port, LED_RED1_Pin, 1);
        HAL_GPIO_WritePin(LED_GREEN1_GPIO_Port, LED_GREEN1_Pin, 0);
        HAL_GPIO_WritePin(LED_AMBER1_GPIO_Port, LED_AMBER1_Pin, 1);
        state_traffic_light = 0;
    }
    else if(counter_traffic_light == TIME_RED_LED - TIME_AMBER_LED){
        HAL_GPIO_WritePin(LED_RED0_GPIO_Port, LED_RED0_Pin, 0);
        HAL_GPIO_WritePin(LED_GREEN0_GPIO_Port, LED_GREEN0_Pin, 1);
        HAL_GPIO_WritePin(LED_AMBER0_GPIO_Port, LED_AMBER0_Pin, 1);
        HAL_GPIO_WritePin(LED_RED1_GPIO_Port, LED_RED1_Pin, 1);
        HAL_GPIO_WritePin(LED_GREEN1_GPIO_Port, LED_GREEN1_Pin, 1);
        HAL_GPIO_WritePin(LED_AMBER1_GPIO_Port, LED_AMBER1_Pin, 0);
        state_traffic_light = 1;
    }
    else if(counter_traffic_light == TIME_RED_LED){
        HAL_GPIO_WritePin(LED_RED0_GPIO_Port, LED_RED0_Pin, 1);
        HAL_GPIO_WritePin(LED_GREEN0_GPIO_Port, LED_GREEN0_Pin, 0);
    }
}

```

```

    HAL_GPIO_WritePin(LED_AMBER0_GPIO_Port, LED_AMBER0_Pin, 1);
    HAL_GPIO_WritePin(LED_RED1_GPIO_Port, LED_RED1_Pin, 0);
    HAL_GPIO_WritePin(LED_GREEN1_GPIO_Port, LED_GREEN1_Pin, 1);
    HAL_GPIO_WritePin(LED_AMBER1_GPIO_Port, LED_AMBER1_Pin, 1);
    state_traffic_light = 2;
}
else if(counter_traffic_light == TIME_RED_LED + TIME_GREEN_LED){
    HAL_GPIO_WritePin(LED_RED0_GPIO_Port, LED_RED0_Pin, 1);
    HAL_GPIO_WritePin(LED_GREEN0_GPIO_Port, LED_GREEN0_Pin, 1);
    HAL_GPIO_WritePin(LED_AMBER0_GPIO_Port, LED_AMBER0_Pin, 0);
    HAL_GPIO_WritePin(LED_RED1_GPIO_Port, LED_RED1_Pin, 0);
    HAL_GPIO_WritePin(LED_GREEN1_GPIO_Port, LED_GREEN1_Pin, 1);
    HAL_GPIO_WritePin(LED_AMBER1_GPIO_Port, LED_AMBER1_Pin, 1);
    state_traffic_light = 3;
}
//CONTROL 7 7SEG-LED
if(counter_7LED <= 0){
    if(state_traffic_light == 0){
        int temp = (TIME_RED_LED - counter_traffic_light)*TRAFFIC_SCAN/1000;
        led_buffer[2] = temp/10;
        led_buffer[3] = temp%10;
        temp -= TIME_AMBER_LED*TRAFFIC_SCAN/1000;
        led_buffer[0] = temp/10;
        led_buffer[1] = temp%10;
    }
    else if(state_traffic_light == 1){
        int temp = (TIME_RED_LED - counter_traffic_light)*TRAFFIC_SCAN/1000;
        led_buffer[2] = temp/10;
        led_buffer[3] = temp%10;
        led_buffer[0] = temp/10;
        led_buffer[1] = temp%10;
    }
    else if(state_traffic_light == 2){
        int temp = (TIME_RED_LED + TIME_GREEN_LED -
            counter_traffic_light)*TRAFFIC_SCAN/1000;
        led_buffer[2] = temp/10;
        led_buffer[3] = temp%10;
        temp += TIME_AMBER_LED*TRAFFIC_SCAN/1000;
    }
}

```

```

        led_buffer[0] = temp/10;
        led_buffer[1] = temp%10;
    }
    else if(state_traffic_light == 3){
        int temp = (TIME_RED_LED + TIME_GREEN_LED + TIME_AMBER_LED -
                    counter_traffic_light)*TRAFFIC_SCAN/1000;
        led_buffer[2] = temp/10;
        led_buffer[3] = temp%10;
        led_buffer[0] = temp/10;
        led_buffer[1] = temp%10;
    }
    counter_7LED = 1000/TRAFFIC_SCAN;
}
counter_7LED--;
counter_traffic_light++;
if(counter_traffic_light >= TIME_RED_LED + TIME_GREEN_LED + TIME_AMBER_LED)
    counter_traffic_light = 0;
}

```

2.7 Exercise 7: Adding code for increasing time duration value for the red LEDs

Your tasks in this exercise are:

- to use the second button to increase the time duration value of the red LEDs
- to use the third button to set the value for the red LEDs.

Please add your report here.

Listing 7: code in mode 1

```

if(flagTransitionLongPress[0] == 1){
    flagTransitionLongPress[0] = 0;
    mode = 2;
    TempConfigure = TIME_AMBER_LED;
    counter_long_press = 0;
    HAL_GPIO_WritePin(LED_RED0_GPIO_Port, LED_RED0_Pin, 1);
    HAL_GPIO_WritePin(LED_GREEN0_GPIO_Port, LED_GREEN0_Pin, 1);
    HAL_GPIO_WritePin(LED_AMBER0_GPIO_Port, LED_AMBER0_Pin, 1);
}

```

```

        HAL_GPIO_WritePin(LED_RED1_GPIO_Port, LED_RED1_Pin, 1);
        HAL_GPIO_WritePin(LED_GREEN1_GPIO_Port, LED_GREEN1_Pin, 1);
        HAL_GPIO_WritePin(LED_AMBER1_GPIO_Port, LED_AMBER1_Pin, 1);
        return;
    }
    if(flagPress[2] == 1){
        flagPress[2] = 0;
        TIME_RED_LED = TempConfigure;
        return;
    }
    if(counter_configure_mode >= 250/TIME_SCAN){
        counter_configure_mode = 0;
        HAL_GPIO_TogglePin(LED_RED0_GPIO_Port, LED_RED0_Pin);
        HAL_GPIO_TogglePin(LED_RED1_GPIO_Port, LED_RED1_Pin);
    }
}

```

2.8 Exercise 8: Adding code for increasing time duration value for the amber LEDs

Your tasks in this exercise are:

- to use the second button to increase the time duration value of the amber LEDs
- to use the third button to set the value for the amber LEDs.

Please add your report here.

Listing 8: code in mode 2

```

if(flagTransitionLongPress[0] == 1){
    flagTransitionLongPress[0] = 0;
    mode = 3;
    TempConfigure = TIME_GREEN_LED;
    counter_long_press = 0;
    HAL_GPIO_WritePin(LED_RED0_GPIO_Port, LED_RED0_Pin, 1);
    HAL_GPIO_WritePin(LED_GREEN0_GPIO_Port, LED_GREEN0_Pin, 1);
    HAL_GPIO_WritePin(LED_AMBER0_GPIO_Port, LED_AMBER0_Pin, 1);
    HAL_GPIO_WritePin(LED_RED1_GPIO_Port, LED_RED1_Pin, 1);
    HAL_GPIO_WritePin(LED_GREEN1_GPIO_Port, LED_GREEN1_Pin, 1);
    HAL_GPIO_WritePin(LED_AMBER1_GPIO_Port, LED_AMBER1_Pin, 1);
}

```

```

        return;
    }
    if(flagPress[2] == 1){
        flagPress[2] = 0;
        TIME_AMBER_LED = TempConfigure;
        return;
    }
    if(counter_configure_mode >= 250/TIME_SCAN){
        counter_configure_mode = 0;
        HAL_GPIO_TogglePin(LED_AMBER0_GPIO_Port, LED_AMBER0_Pin);
        HAL_GPIO_TogglePin(LED_AMBER1_GPIO_Port, LED_AMBER1_Pin);
    }

```

2.9 Exercise 9: Adding code for increasing time duration value for the green LEDs

Your tasks in this exercise are:

- to use the second button to increase the time duration value of the green LEDs
- to use the third button to set the value for the green LEDs.

Please add your report here.

Listing 9: code in mode 3

```

if(flagTransitionLongPress[0] == 1){
    flagTransitionLongPress[0] = 0;
    mode = 0;
    return;
}
if(flagPress[2] == 1){
    flagPress[2] = 0;
    TIME_GREEN_LED = TempConfigure;
    counter_long_press = 0;
    counter_traffic_light = 0;
    return;
}
if(counter_configure_mode >= 250/TIME_SCAN){
    counter_configure_mode = 0;

```

```
    HAL_GPIO_TogglePin(LED_GREEN0_GPIO_Port, LED_GREEN0_Pin);  
    HAL_GPIO_TogglePin(LED_GREEN1_GPIO_Port, LED_GREEN1_Pin);  
}
```

2.10 Exercise 10: To finish the project

Your tasks in this exercise are:

- To integrate all the previous tasks to one final project
- To create a video to show all features in the specification
- To add a report to describe your solution for each exercise.
- To submit your report and code on the BKeL

Listing 10: function of mode

```
void auto_mode(void){  
if(mode != 0) return;  
if(flagTransitionLongPress[0] == 1){  
    flagTransitionLongPress[0] = 0;  
    mode = 1;  
    counter_traffic_light = 0;  
    counter_7LED = 0;  
    TempConfigure = TIME_RED_LED;  
    HAL_GPIO_WritePin(LED_RED0_GPIO_Port, LED_RED0_Pin, 1);  
    HAL_GPIO_WritePin(LED_GREEN0_GPIO_Port, LED_GREEN0_Pin, 1);  
    HAL_GPIO_WritePin(LED_AMBER0_GPIO_Port, LED_AMBER0_Pin, 1);  
    HAL_GPIO_WritePin(LED_RED1_GPIO_Port, LED_RED1_Pin, 1);  
    HAL_GPIO_WritePin(LED_GREEN1_GPIO_Port, LED_GREEN1_Pin, 1);  
    HAL_GPIO_WritePin(LED_AMBER1_GPIO_Port, LED_AMBER1_Pin, 1);  
    return;  
}  
if(flagPress[0] == 1 || flagPress[1] == 1 || flagPress[2] == 1){flagPress[0] = 0;  
    flagPress[1] = 0; flagPress[2] = 0;}  
if(flagLongPress[0] == 1 || flagLongPress[1] == 1 || flagLongPress[2] == 1){  
    flagLongPress[0] = 0; flagLongPress[1] = 0; flagLongPress[2] = 0;  
}  
if(TIME_RED_LED <= TIME_AMBER_LED){
```

```

    HAL_GPIO_TogglePin(LED_RED0_GPIO_Port, LED_RED0_Pin);
    HAL_GPIO_TogglePin(LED_GREEN0_GPIO_Port, LED_GREEN0_Pin);
    HAL_GPIO_TogglePin(LED_AMBER0_GPIO_Port, LED_AMBER0_Pin);
    HAL_GPIO_TogglePin(LED_RED1_GPIO_Port, LED_RED1_Pin);
    HAL_GPIO_TogglePin(LED_GREEN1_GPIO_Port, LED_GREEN1_Pin);
    HAL_GPIO_TogglePin(LED_AMBER1_GPIO_Port, LED_AMBER1_Pin);
    return;
}

traffic_light();
}

#define TIME_COUNT_IN_LONG_PRESS 100
int counter_long_press = 0;
void configure_mode(void){
    if(mode == 0) return;
    switch(mode){
    case 1:
        if(flagTransitionLongPress[0] == 1){
            flagTransitionLongPress[0] = 0;
            mode = 2;
            TempConfigure = TIME_AMBER_LED;
            counter_long_press = 0;
            HAL_GPIO_WritePin(LED_RED0_GPIO_Port, LED_RED0_Pin, 1);
            HAL_GPIO_WritePin(LED_GREEN0_GPIO_Port, LED_GREEN0_Pin, 1);
            HAL_GPIO_WritePin(LED_AMBER0_GPIO_Port, LED_AMBER0_Pin, 1);
            HAL_GPIO_WritePin(LED_RED1_GPIO_Port, LED_RED1_Pin, 1);
            HAL_GPIO_WritePin(LED_GREEN1_GPIO_Port, LED_GREEN1_Pin, 1);
            HAL_GPIO_WritePin(LED_AMBER1_GPIO_Port, LED_AMBER1_Pin, 1);
            return;
        }
        if(flagPress[2] == 1){
            flagPress[2] = 0;
            // mode = 0;
            TIME_RED_LED = TempConfigure;
            return;
        }
        if(counter_configure_mode >= 250/TIME_SCAN){
            counter_configure_mode = 0;

```

```

        HAL_GPIO_TogglePin(LED_RED0_GPIO_Port, LED_RED0_Pin);
        HAL_GPIO_TogglePin(LED_RED1_GPIO_Port, LED_RED1_Pin);
    }
    break;
case 2:
    if(flagTransitionLongPress[0] == 1){
        flagTransitionLongPress[0] = 0;
        mode = 3;
        TempConfigure = TIME_GREEN_LED;
        counter_long_press = 0;
        HAL_GPIO_WritePin(LED_RED0_GPIO_Port, LED_RED0_Pin, 1);
        HAL_GPIO_WritePin(LED_GREEN0_GPIO_Port, LED_GREEN0_Pin, 1);
        HAL_GPIO_WritePin(LED_AMBER0_GPIO_Port, LED_AMBER0_Pin, 1);
        HAL_GPIO_WritePin(LED_RED1_GPIO_Port, LED_RED1_Pin, 1);
        HAL_GPIO_WritePin(LED_GREEN1_GPIO_Port, LED_GREEN1_Pin, 1);
        HAL_GPIO_WritePin(LED_AMBER1_GPIO_Port, LED_AMBER1_Pin, 1);
        return;
    }
    if(flagPress[2] == 1){
        flagPress[2] = 0;
        TIME_AMBER_LED = TempConfigure;
//        mode = 0;
        return;
    }
    if(counter_configure_mode >= 250/TIME_SCAN){
        counter_configure_mode = 0;
        HAL_GPIO_TogglePin(LED_AMBER0_GPIO_Port, LED_AMBER0_Pin);
        HAL_GPIO_TogglePin(LED_AMBER1_GPIO_Port, LED_AMBER1_Pin);
    }
    break;
case 3:
    if(flagTransitionLongPress[0] == 1){
        flagTransitionLongPress[0] = 0;
        mode = 0;
        return;
    }
    if(flagPress[2] == 1){
        flagPress[2] = 0;

```

```

//      mode = 0;
      TIME_GREEN_LED = TempConfigure;
      counter_long_press = 0;
      counter_traffic_light = 0;
      return;
    }
    if(counter_configure_mode >= 250/TIME_SCAN){
        counter_configure_mode = 0;
        HAL_GPIO_TogglePin(LED_GREEN0_GPIO_Port, LED_GREEN0_Pin);
        HAL_GPIO_TogglePin(LED_GREEN1_GPIO_Port, LED_GREEN1_Pin);
    }
    break;
}
led_buffer[2] = 0;
led_buffer[3] = mode;
int temp = TempConfigure*TRAFIC_SCAN/1000;
led_buffer[0] = temp/10;
led_buffer[1] = temp%10;

if(flagPress[0] == 1){
    flagPress[0] = 0;
    TempConfigure -= 1000/TRAFIC_SCAN;
    if(TempConfigure <= 0) TempConfigure = 99000/TRAFIC_SCAN;
}

if(flagPress[1] == 1){
    flagPress[1] = 0;
    TempConfigure += 1000/TRAFIC_SCAN;
    if(TempConfigure > 99000/TRAFIC_SCAN) TempConfigure = 1000/TRAFIC_SCAN;
}

if(flagLongPress[1] == 1){
    flagLongPress[1] = 0;
    if(counter_long_press >= TIME_COUNT_IN_LONG_PRESS /TIME_SCAN){
        TempConfigure += 1000/TRAFIC_SCAN;
        if(TempConfigure > 99000/TRAFIC_SCAN) TempConfigure = 1000/TRAFIC_SCAN;
        counter_long_press = 0;
    }
}

```

```
    else counter_long_press++;  
  }  
  counter_configure_mode++;  
}
```

Listing 11: code in while

```
init_running();  
setTimer(0, 10);  
setTimer(1, 10);  
while (1)  
{  
    if(istimer_flag(0) == 1){  
        fsm_button();  
        auto_mode();  
        configure_mode();  
        setTimer(0, TRAFIC_SCAN);  
    }  
    if(istimer_flag(1) == 1){  
        if(index_led > 3) index_led = 0;  
        update7SEG(index_led++);  
        setTimer(1, 50);  
    }  
}
```

References